

现代编程思想

命令式编程

月兔公开课课程组

函数式编程

- 到此为止，我们介绍的编程知识可以归类于函数式编程的范畴
 - 对每一个输入，有着固定的输出
 - 对于标识符，我们可以直接用它所对应的值进行替代——引用透明性
- 开发实用的程序，我们需要一些计算之外的“副作用”
 - 进行输入输出
 - 修改内存中的数据等
 - 这些副作用可能导致多次执行的结果不一致

引用透明性

- 我们可以定义如下数据绑定和函数

```
1. let x : Int = 1 + 1
2. fn square(x : Int) -> Int { x * x }
3. let z : Int = square(x) // 4
```

- 我们可以将 `square` 与 `x` 直接用对应的值替换而不改变结果

```
1. let z : Int = { 2 * 2 } // 4
```

- 引用透明性可以易于理解

命令

- 函数 `println` 允许我们输出一个字符串，例如 `println("hello moonbit")`
- 月兔中可以通过 `init` 代码块来定义初始化指令

```
1. fn init {  
2.     println("hello moonbit")  
3. }
```

⚙ Debug

```
fn main {  
    println("hello moonbit")  
}
```

hello moonbit

命令与副作用

- 输出命令可能会破坏引用透明性

```
fn square(x : Int) -> Int { x * x }  
fn init {  
  let x : Int = {  
    println("hello moonbit") // <-- 我们首先执行命令，进行输出  
    1 + 1 // <-- 之后，我们以表达式块最后的值作为表达式块的值  
  }  
  let z : Int = square(x) // 4, 输出一次  
}
```

hello moonbit

命令与副作用

- 我们不一定可以放心替换，因此会增大程序理解难度

```
fn init {  
  let z : Int = {  
    println("hello moonbit"); // <-- 进行了第一次输出  
    1 + 1 // <-- 获得值: 2  
  } * {  
    println("hello moonbit"); // <-- 进行了第二次输出  
    1 + 1 // <-- 获得值: 2  
  } // 4, 输出两次  
}
```

```
hello moonbit  
hello moonbit
```

单值类型

- 我们之前已经介绍过单值类型 `Unit`
 - 它仅有一个值: `()`
- 以 `Unit` 为运算结果类型的函数或命令一般有副作用
 - `fn println(String) -> Unit`
- 命令的类型也是单值类型

```
1. fn do_nothing() -> Unit {  
2.   let _x = 0 // 结果为单值类型, 符合函数定义  
3. }
```

变量

- 在月兔中，我们可以在代码块中用 `let mut` 定义临时变量

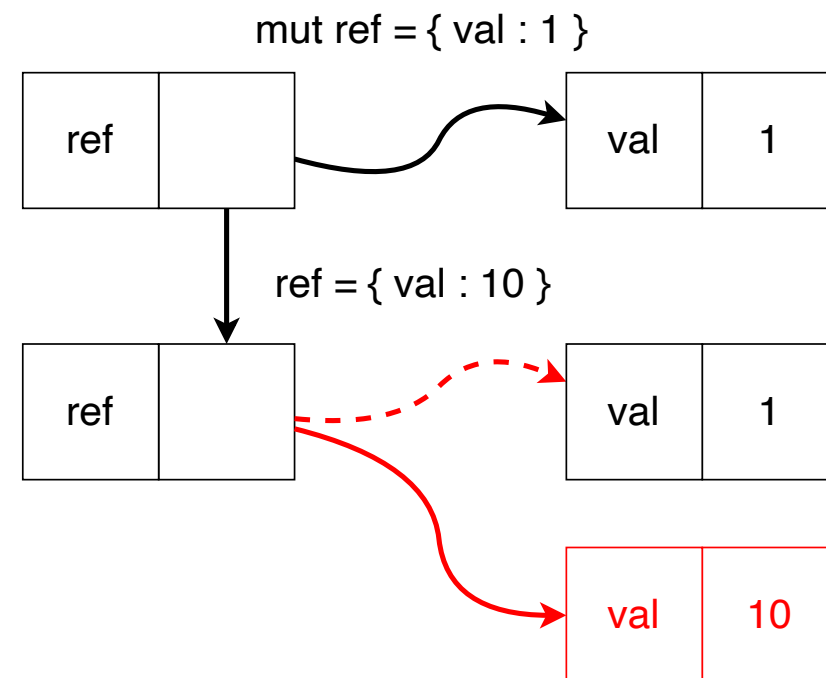
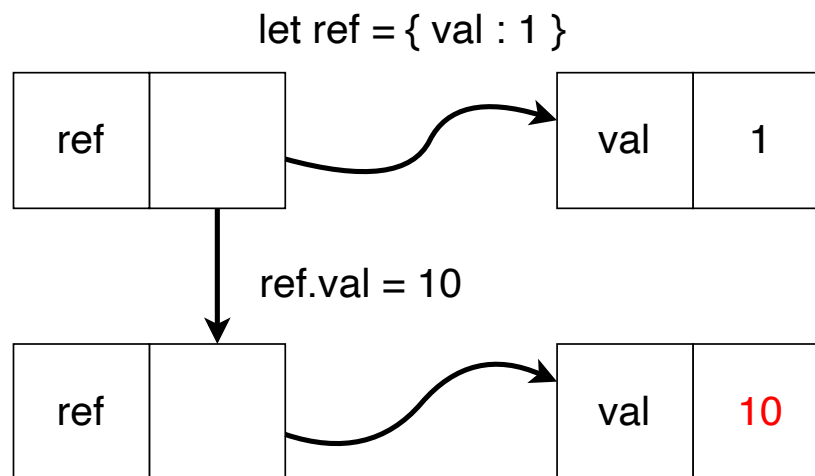
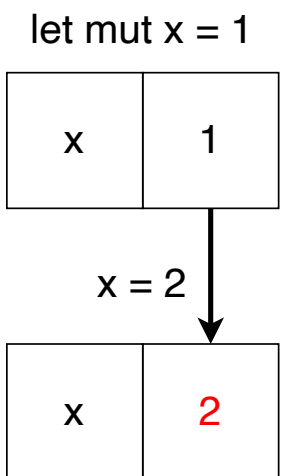
```
1. test {  
2.     let mut x = 1  
3.     x = 10 // 赋值操作是一个命令  
4. }
```

- 在月兔中，结构体的字段默认不可变，我们也允许可变的字段，需要用 `mut` 标识

```
1. struct Ref[T] { mut val : T }  
2.  
3. fn init {  
4.     let ref : Ref[Int] = { val : 1 } // ref 本身只是一个数据绑定  
5.     ref.val = 10 // 我们可以修改结构体的字段  
6.     println(ref.val.to_string()) // 输出 10  
7. }
```

变量

- 我们可以将带有可变字段的结构体看作是引用



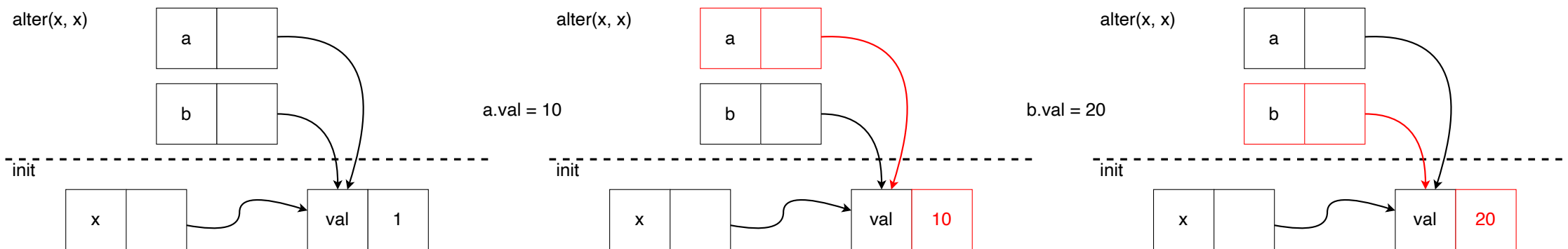
别名

- 指向相同的可变数据结构的两个标识符可以看作是别名

```
1. fn alter(a : Ref[Int], b : Ref[Int]) -> Unit {  
2.   a.val = 10  
3.   b.val = 20  
4. }  
5.  
6. fn init {  
7.   let x : Ref[Int] = { val : 1 }  
8.   alter(x, x)  
9.   println(x.val.to_string()) // x.val的值将会被改变两次  
10. }
```

别名

- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理

循环

- 多次执行同一段代码
- `while`, `for`, `loop`
- 循环结构也是一个表达式

while 循环

```
1. let mut i = 0
2. while i < 3 {
3.     println("hello")
4.     i = i + 1
5. }
```

 // 重复输出3次

```
fn init {
    let mut i = 0
    while i < 3 {
        println("hello")
        i = i + 1
    }
}
```

```
hello
hello
hello
```

while 循环

```
1. while <条件表达式> {  
2.     <循环体>  
3. }
```

- 当条件为真，执行循环体，并回到条件表达式进入下一次判断
- 当条件为假，退出循环体

while 循环

- 利用变量设定循环条件

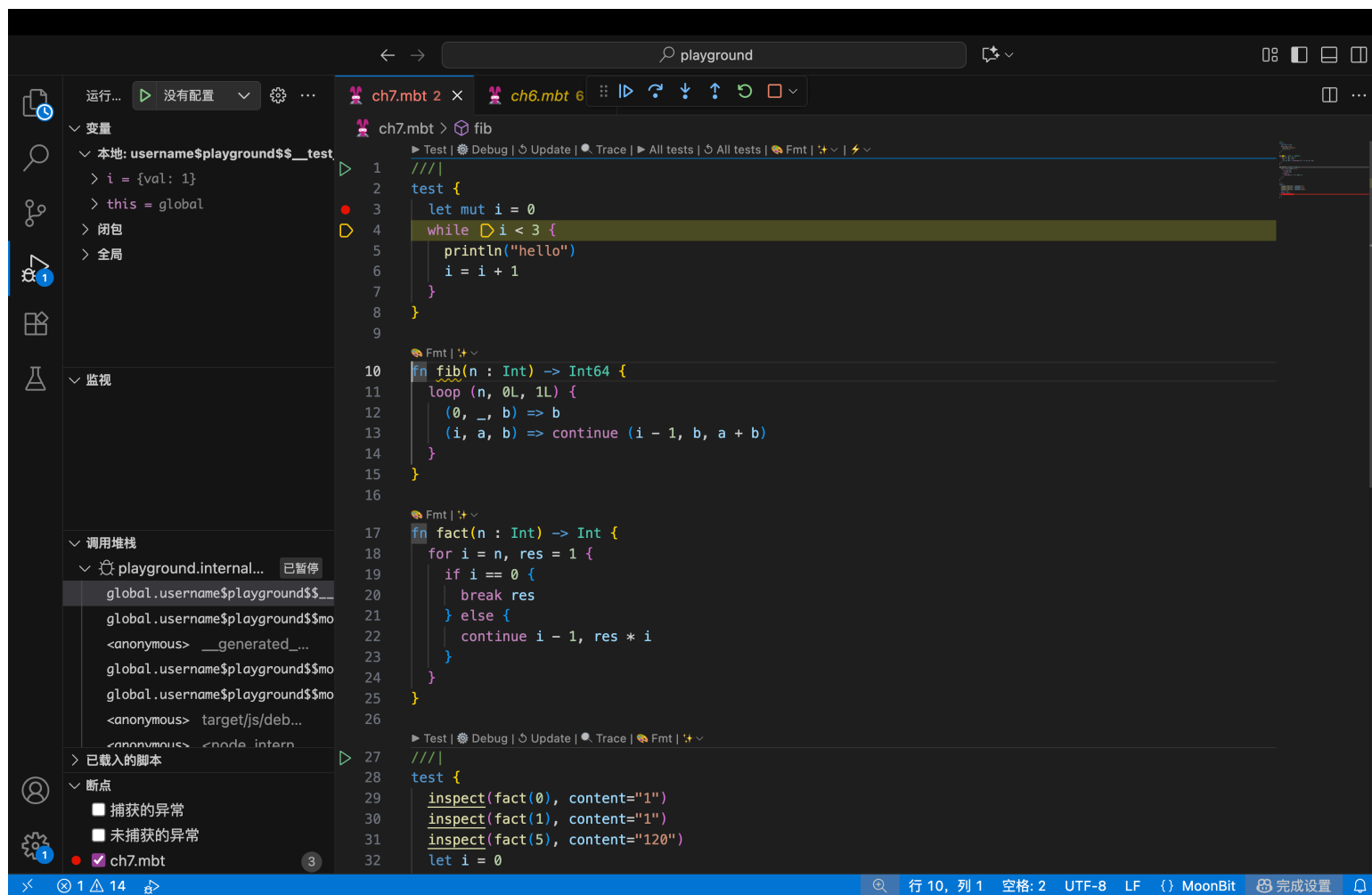
```
1. <定义变量及初始值>
2. while <针对变量判断是否继续循环> {
3.     <需要重复执行的命令>
4.     <对变量进行更新>
5. }
```

- 例如，我们可以反复执行3次输出操作

```
1. let mut i = 0
2. while i < 3 {
3.     println("hello")
4.     i = i + 1
5. } // 重复输出3次
```

调试器

- 月兔的调试器允许我们在运行中看到实时的运行数据，更好理解运行过程



while 循环的 else 块

- 在 while 循环的末尾可以加上一个 else 块
- 循环条件为假时，会执行一次 else 块
- else 块的值即是整个 while 循环表达式的值
- 例子：

```
1. let mut i = 0
2. let n = while i < 3 {
3.     println("hello")
4.     i = i + 1
5. } else {
6.     println("end")
7.     i
8. }
```

循环与递归

- 事实上，循环与递归是等价的

```
1. let mut <变量> = <初始值>
2. while <判断是否继续循环> {
3.     <需要重复执行的命令>
4.     <对变量进行更新>
5. }
```

- 利用可变变量的情况下可以写成

```
1. fn recurse(<参数>) -> Unit {
2.     if <判断是否继续循环> {
3.         <需要重复执行的命令>
4.         recurse(<更新后的参数>)
5.     } else { () }
6. }
7. recurse(<初始值>)
```

循环与递归

- 例如下述两段代码执行效果相同

```
1. let mut i = 0
2. while i < 2 {
3.     println("Hello!")
4.     i = i + 1
5. }
```

```
1. fn recurse(i : Int) -> Unit {
2.     if i < 2 {
3.         println("Hello!")
4.         recurse(i + 1)
5.     } else { () }
6. }
7. recurse(0)
```

循环流的控制

- 循环的时候，可以提前中止循环，或是跳过后续命令的执行
 - `break` 指令可以中止循环
 - `continue` 指令可以跳过后续运行，直接进入下一次循环

```
1. fn print_first_3() -> Unit {  
2.   let mut i = 0  
3.   while i < 10 {  
4.     if i == 3 {  
5.       break // 跳过从3开始的情况  
6.     } else {  
7.       println(i.to_string())  
8.     }  
9.     i = i + 1  
10.  }  
11. }
```

循环流的控制

- 循环的时候，可以提前中止循环，或是跳过后续命令的执行
 - `break` 指令可以中止循环
 - `continue` 指令可以跳过后续运行，直接进入下一次循环

```
1. fn print_skip_3() -> Unit {  
2.     let mut i = 0  
3.     while i < 10 {  
4.         i = i + 1  
5.         if i == 3 {  
6.             continue // 跳过3  
7.         }  
8.         println(i.to_string())  
9.     }  
10. }
```

for 循环

```
1. for <定义变量及初始值>; <判断是否继续循环>; <对变量进行更新> {  
2.     <需要重复执行的命令>  
3. }
```

等价于

```
1. <定义变量及初始值>  
2. while <判断是否继续循环> {  
3.     <需要重复执行的命令>  
4.     <对变量进行更新>  
5. }
```

for 循环

```
1. for i = 0; i < 3; i = i + 1 {  
2.     println("hello")  
3. }
```

- for 循环的变量定义无需用 `let mut`
- for 循环限制了对循环变量的修改

```
1. for i = 0; i < 3; i = i + 1 {  
2.     println("hello")  
3.     i = i + 1 // 不合法! 循环变量不可在这里更改  
4. }
```

for 循环

- 不使用变量更新语句，也可以更新变量。例如

```
1. for i = 0 {  
2.     if i < 3 {  
3.         println("hello")  
4.         continue i + 1  
5.     } else {  
6.         break  
7.     }  
8. }
```

- 这是函数式循环的典型用例。

函数式循环

- 函数式 loop 循环和函数式 for 循环中使用
- 不显式为变量赋值
- 循环中使用 `break <表达式>` 退出循环，并得到一个值
- 循环中使用 `continue <表达式>` 带着一个值进入下一轮循环

函数式 for 循环

```
1. fn fact(n : Int) -> Int {  
2.   for i = n, res = 1 {           // 初始化循环变量  
3.     if i == 0 {  
4.       break res                 // 将 res 作为整个循环的值  
5.     } else {  
6.       continue i - 1, res * i // 带着更新后的循环变量进入下一轮循环  
7.     }  
8.   }  
9. }  
10.  
11. test {  
12.   inspect(fact(5), content="120")  
13. }
```

loop 循环

- 函数式 for 循环结合模式匹配

```
1. loop <初始值> {  
2.     <模式1> => <表达式1>  
3.     <模式2> => <表达式2>  
4.     ...  
5. }
```

等价于

```
1. for <变量> = <初始值> {  
2.     match <变量> {  
3.         <模式1> => <表达式1>  
4.         <模式2> => <表达式2>  
5.         ...  
6.     }  
7. }
```

loop 循环

```
1. fn fact_with_loop(n : Int) -> Int {  
2.     loop (n, 1) {                                // 初始化循环变量  
3.         // 对二元组进行模式匹配  
4.         (0, res) => break res                    // 将 res 作为整个循环的值  
5.         (i, res) => continue (i - 1, res * i) // 带着更新后的循环变量进入下一轮循环  
6.     }  
7. }  
8.  
9. test {  
10.     inspect(fact_with_loop(5), content="120")  
11. }
```

月兔的检查

- 月兔会检查一个变量是否被修改，可以避免出现循环忘记更新变量

```

}
Error (warning): The mutability of 'i' is never used, try remove `mut`. (E0015)
Int
fn init { 查看问题 (⌘F8) 快速修复... (⌘.)
  let mut i = 0
  while i < 5 {
    println("hello")
  }
}

```

- 月兔也会检查函数返回结果是否与类型声明相同，可以避免错误的返回类型声明

```

fn plus_one(n : Int) -> Unit {
  n + 1
}
Expr Type Mismatch
  has type : Int
  wanted   : Unit (E4014)
//
te Int
查看问题 (⌘F8) 没有可用的快速修复

```

- 使用场景广泛
 - 直接操作程序外环境，如硬件等
 - 一些情况下性能更好，如随机访问数组等
 - 可以构建部分复杂数据结构
 - 重复利用空间（原地修改）
- 可变数据并不总是与引用透明性冲突

```
1. fn fib_mut(n : Int) -> Int { // 对于相同输入，总是有相同输出
2.   let mut acc1 = 0; let mut acc2 = 1; let mut i = 0
3.   while i < n {
4.     let t = acc1 + acc2; acc1 = acc2; acc2 = t
5.     i = i + 1
6.   }
7.   acc1
8. }
```

总结

本章节初步接触了命令式编程，包括

- 如何使用命令
- 如何使用变量
- 如何使用循环等